



Software Testing and Quality Assurance (05302438)

Yadagiri Rama Deepak,
Assistant Professor
Computer Science & Applications



Module-6

Software Quality Assurance

- Elements of Software Quality Assurance(SQA),
- SQA-need,
- Activities,
- Tasks,
- Goals,
- Metrics,
- Statistical software quality assurance,
- Six sigma for software engineering,
- SQA planning and standards - International Organization for Standardization [ISO] quality standards.



Elements of Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a set of systematic activities that ensure software processes and products conform to defined quality standards and requirements throughout the software development lifecycle (SDLC).



Definition:

SQA is a **planned and systematic set of activities** necessary to provide adequate confidence that a software product conforms to **requirements, standards, and procedures.**



Key Elements of SQA:

Element	Description
1. Software Engineering Standards	Use of industry standards (e.g., ISO 25010, IEEE, CMMI) to define software processes and product quality.
2. Formal Technical Reviews (FTR)	Peer reviews of design, code, or documents to identify errors early. Helps improve the quality of work products.
3. Testing Control Measures	Defining test strategies, test plans, test cases, and metrics to control the testing process.
4. Software Configuration Management (SCM)	Tracks and controls changes to the software to maintain integrity and traceability.
5. Documentation Control	Ensures proper versioning, updates, and availability of documents like SRS, design specs, user manuals, etc.
6. Training and Certification	Ongoing training of the development and QA teams to keep up with quality processes, tools, and best practices.
7. Problem Reporting and Corrective Action (PRACA)	Mechanisms for logging defects, tracking issues, and ensuring root cause analysis and corrective action.
8. Tools and Automation	Use of tools for testing, version control, static code analysis, bug tracking, etc., to improve consistency and efficiency.
9. Metrics Collection and Analysis	Collecting data (defect density, code coverage, etc.) to evaluate process performance and product quality.
10. Audits and Quality Assurance Plan	Regular internal audits and following a formal QA plan to ensure compliance with organizational and project-level quality goals.



Goals of SQA:

Ensure conformance to requirements.

- Prevent **defects** rather than detect them later.
- Increase **reliability and maintainability**.
- Promote **customer satisfaction**.



SQA Activities Across SDLC Phases:

SDLC Phase	SQA Activities
Requirements	Requirement reviews, validation
Design	Design walkthroughs, standards compliance
Coding	Code reviews, static analysis
Testing	Test planning, execution, defect tracking
Deployment	Release audits, performance validation



Best Practices for SQA:

- Involve QA from the **beginning of the project**.
- Use **automated testing and CI/CD** pipelines.
- Encourage **cross-functional collaboration** between developers, testers, and managers.
- Apply **risk-based testing** and prioritize critical modules.



Metrics

Software metrics are quantitative measures that help assess the quality of software, the efficiency of testing, and the effectiveness of the overall software development and QA process.

Metrics

1. Types of Metrics

(A) Product Metrics

Measure the characteristics of the **software product**.

Examples:

Size metrics: Lines of Code (LOC), Function Points.

Complexity metrics: Cyclomatic complexity.

Quality metrics: Defect density, reliability, maintainability.

(B) Process Metrics

Measure the efficiency and performance of the **software process**.

Examples:

Defect removal efficiency (DRE).

Effort spent on reviews vs. testing.

Testing efficiency.

Mean time to failure (MTTF).

Metrics

(C) Project Metrics

Track progress and management aspects of a **specific project**.

Examples:

- Effort (person-hours).

- Cost variance.

- Schedule variance.

- Test case execution rate.

Metrics in STQA act as a **thermometer of quality**—they measure how well the process, product, and testing are performing and guide improvements.

Metrics

2. Testing-Specific Metrics

Defect Density = Number of defects / Size of software (e.g., per KLOC).

Defect Leakage = Defects found after release / Total defects.

Defect Removal Efficiency (DRE) = Defects removed before release / Total defects.

Test Case Effectiveness = Defects detected by test cases / Total defects.

Test Coverage = (Number of items covered / Total items) × 100.

Mean Time to Failure (MTTF) = Average time software runs before failure.

Cost of Quality (CoQ) = Prevention + Appraisal + Failure costs.

3. Importance of Metrics in STQA

Provides **objective measurement** of quality.

Helps in **tracking progress** of testing activities.

Supports **process improvement**.

Identifies **weak areas** in development/testing.

Provides **data for management decisions** (release readiness, cost control).



Statistical software quality assurance

Statistical Software Quality Assurance (SSQA) is the use of **statistical methods and quantitative techniques** to measure, monitor, control, and improve the quality of software.

It applies principles of **statistical quality control (SQC)**—commonly used in manufacturing—to **software engineering**, ensuring that defects are minimized and quality is objectively measured.

Statistical software quality assurance

1. Key Concepts of SSQA

- **Defect Data Collection** → Gathering data about defects (type, severity, frequency, phase detected).
- **Statistical Analysis** → Applying mathematical/statistical techniques to analyze defects.
- **Control Charts & Trends** → Monitoring whether the process is stable or has variations.
- **Reliability Growth Models** → Predicting how software reliability improves as defects are removed.
- **Defect Prediction Models** → Using past data to estimate future defect rates.

Statistical software quality assurance

2. SSQA Techniques

- **Pareto Analysis (80/20 Rule):**
 - 80% of problems come from 20% of causes.
 - Helps prioritize defect-prone modules.
- **Control Charts:**
 - Used to track defect density, defect arrival rate, and failure rate.
 - Helps determine whether the process is **in control**.
- **Cause-and-Effect Diagrams (Ishikawa / Fishbone):**
 - Identify root causes of defects.
- **Scatter Plots / Regression Analysis:**
 - Study the relationship between defects and effort, time, or size.
- **Defect Density Analysis:**
 - Defects per KLOC (Kilo Lines of Code) or per Function Point.

Statistical software quality assurance

3. SSQA Process

- **Collect data** (defects, failures, costs, test results).
- **Categorize data** (severity, module, development phase).
- **Apply statistical techniques** (charts, distributions, models).
- **Interpret results** (identify high-risk modules, process weaknesses).
- **Implement improvements** (process changes, more reviews, targeted testing).

Statistical software quality assurance

4. Benefits of SSQA

- Objective and data-driven quality evaluation.
- Early identification of **defect-prone modules**.
- Supports **predictive quality management**.
- Reduces **cost of poor quality** by targeting problem areas.
- Enhances **software reliability and customer satisfaction**.

SSQA = Applying statistical methods (like defect density, control charts, reliability models) to monitor, predict, and improve software quality.
It ensures that **quality is measurable, predictable, and continuously improved.**

Six sigma for software engineering

What is Six Sigma?

Six Sigma is a **data-driven quality management methodology** focused on **reducing defects and variability** in processes.

Originally developed by **Motorola and popularized by GE**, it aims for **near-perfect quality**:

3.4 defects per million opportunities (DPMO).

When applied to **software engineering**, Six Sigma ensures that **software processes and products** are efficient, predictable, and meet customer requirements with minimal defects.

Six sigma for software engineering

1. Core Principles of Six Sigma

- **Customer Focus** → Define quality from the customer's perspective.
- **Data-Driven Decision Making** → Use metrics and measurements, not assumptions.
- **Defect Reduction** → Strive for near-zero defects in software.
- **Continuous Improvement** → Ongoing monitoring and optimization of processes.
- **Process Control** → Standardization to minimize variability in coding, testing, and delivery.

Six sigma for software engineering

2. Six Sigma Methodologies in Software Engineering

(A) DMAIC Model (for improving existing processes)

- **Define** → Identify customer requirements & problem areas in software.
- **Measure** → Collect data (defects, rework rate, test coverage).
- **Analyze** → Identify root causes (e.g., bad coding practices, poor requirement clarity).
- **Improve** → Implement solutions (automation, better reviews, refactoring).
- **Control** → Monitor metrics to ensure sustained improvements.

(B) DMADV Model (for new software development)

- **Define** → Customer needs & software goals.
- **Measure** → Define quality metrics & success factors.
- **Analyze** → Evaluate alternative designs or architectures.
- **Design** → Develop high-quality, defect-minimized software.
- **Verify** → Validate that design meets customer expectations.

Six sigma for software engineering

3. Six Sigma Metrics for Software

- **Defect Density** = Defects per KLOC or Function Point.
- **Defect Leakage** = Defects found after release / Total defects.
- **Defect Removal Efficiency (DRE)** = % defects removed before release.
- **Rework Effort Percentage** = Effort spent fixing issues vs. total effort.
- **Cycle Time** = Time taken for each development/testing phase.



Six sigma for software engineering

4. Benefits of Six Sigma in Software Engineering

- Reduces defects and rework.
- Improves **software reliability and performance**.
- Enhances **customer satisfaction**.
- Brings **predictability** in cost, schedule, and quality.
- Encourages a **culture of continuous improvement**.

5. Challenges

- Measuring defects in software is less tangible than in manufacturing.
- Requires **strong data collection and statistical analysis**.
- Cultural resistance from developers/testers.

Six Sigma in Software Engineering = Applying DMAIC/DMADV with metrics like defect density and DRE to achieve near-zero defects and high customer satisfaction.



SQA planning and standards - International Organization for Standardization [ISO] quality standards

1. Software Quality Assurance (SQA) Planning

SQA planning is the process of **defining activities, standards, tools, techniques, and responsibilities** needed to ensure that software meets its quality objectives.

Key Elements of an SQA Plan (SQAP):

1. **Purpose & Scope** → Define project objectives, quality goals, and boundaries.
2. **Standards & Procedures** → Specify coding standards, testing standards, and documentation practices.
3. **Quality Goals & Metrics** → Define measurable quality attributes (e.g., reliability, usability, performance).
4. **Reviews & Audits** → Plan for inspections, walkthroughs, and quality audits.
5. **Testing Strategy** → Define unit, integration, system, and acceptance testing.
6. **Defect Tracking** → Procedures for defect reporting and corrective actions.
7. **Responsibilities** → Assign roles for developers, testers, QA team, and project managers.
8. **Tools & Automation** → Specify test management, bug tracking, and CI/CD tools.
9. **Risk Management** → Identify quality-related risks and mitigation strategies.
10. **Reporting & Documentation** → Define reporting formats for progress and quality metrics.



SQA planning and standards - International Organization for Standardization [ISO] quality standards

2. Importance of Standards in SQA

- Ensure **consistency** in software development.
- Provide a **benchmark for quality** across projects.
- Improve **maintainability and interoperability**.
- Enable **international recognition** of software products.
- Reduce defects and improve customer satisfaction.



SQA planning and standards - International Organization for Standardization [ISO] quality standards

3. ISO Standards for Software Quality

(A) ISO 9000 Series (Generic Quality Management Standards)

- **ISO 9001:** Specifies requirements for a quality management system (QMS).
- Applied to software companies to ensure **process quality**.
- Focuses on:
 - Documentation of processes.
 - Continuous improvement.
 - Customer satisfaction.

SQA planning and standards - International Organization for Standardization [ISO] quality standards

(B) ISO/IEC 9126 → Replaced by ISO/IEC 25010 (Software Product Quality Standards)

Defines **software quality characteristics** and **sub-characteristics**.

ISO/IEC 25010 Quality Model:

1. **Functional Suitability** – Does it meet requirements?
2. **Performance Efficiency** – Speed, resource usage.
3. **Compatibility** – Interoperability with other systems.
4. **Usability** – User-friendly, accessible.
5. **Reliability** – Fault tolerance, recoverability.
6. **Security** – Confidentiality, integrity, authentication.
7. **Maintainability** – Modularity, reusability, testability.
8. **Portability** – Adaptability across environments.

SQA planning and standards - International Organization for Standardization [ISO] quality standards

(C) ISO/IEC 12207 (Software Life Cycle Processes)

- Framework for **software development and maintenance processes**.
- Covers:
 - Development (requirements, design, coding, testing).
 - Maintenance (bug fixes, enhancements).
 - Supporting processes (documentation, configuration management, QA).

(D) ISO/IEC 15504 (SPICE – Software Process Improvement and Capability Determination)

- Provides a **maturity model** for evaluating and improving software processes.
- Similar to **CMMI**, used to assess process capability levels.



SQA planning and standards - International Organization for Standardization [ISO] quality standards

4. Benefits of ISO Standards in SQA

- Provides **global recognition** of quality.
- Ensures **compliance with best practices**.
- Helps in **contractual obligations** (many clients demand ISO-certified vendors).
- Improves **efficiency and reduces cost of poor quality**.
- Enhances **customer trust** and market competitiveness.

SQA planning defines how quality will be ensured, while **ISO standards (ISO 9001, ISO/IEC 25010, ISO/IEC 12207, and ISO/IEC 15504)** provide **internationally recognized frameworks** to measure, monitor, and improve software process and product quality.

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

